

Spring Boot Shopping Cart Web App Deployment

15 min read · May 11, 2024



Manisha Acharya

Follow

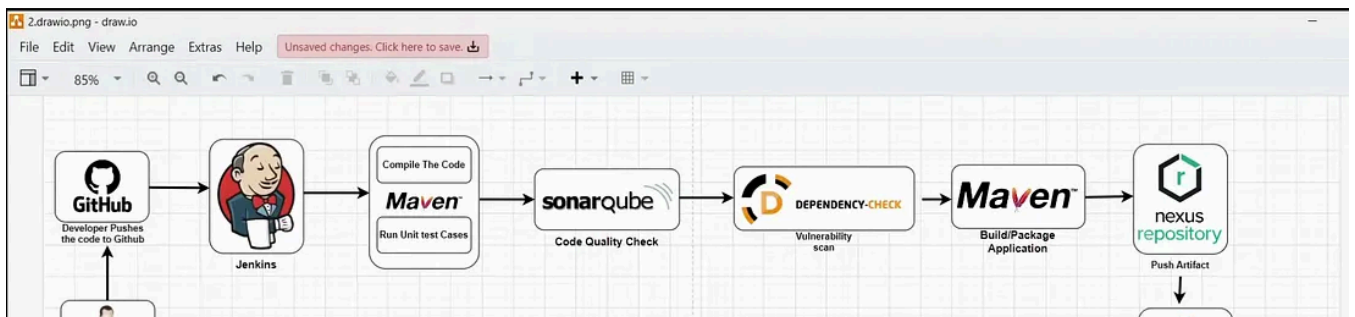


Listen



Share

We're getting ready to launch our new e-commerce app! We're using Jenkins to automate building and testing our software (CI/CD), and deploying it using Docker containers on a Kubernetes cluster. Also you will find the live error-solving inputs for the pipeline failures.



Open in app

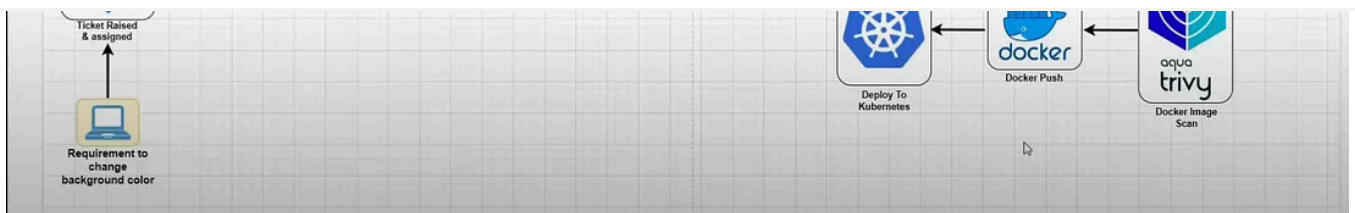
Sign up

Sign in

Medium



Search



Tools we used

- Git
- Jenkins

- Maven
- SonarQube
- OWASP Dependency-Check
- Nexus Repository
- Docker
- Trivy
- Kubernetes

Steps to follow:-

1. Launch Ubuntu Instance t2-large in AWS
2. Install Jenkins
3. Install Sonarqube in the Sonar Instance
4. Install Nexus in the Nexus EC2 instance
5. Install Plugins & Configure them
6. Integrate Sonarqube & Nexus servers with Jenkins
7. Create Pipeline in Jenkins
8. Kubernetes Cluster setup
9. Create Namespace, Service, Role, Bind Role to service, generate token
10. Build & tag the Docker image
11. Integrate Trivy with Jenkins
12. Create a stage to push the Docker image to DockerHub
13. Create a stage to deploy the application to the K8S cluster

GitHub link: <https://github.com/manishaacharya111/Ekart/tree/main>

Let us go ahead and start the demonstration.

1. Launch Ubuntu Instance t2-large in AWS

Launch 3 AWS *t2-large* Instance with the image as *Ubuntu 24.04 LTS*. You can create a new key pair or use an existing one. Enable HTTP and HTTPS settings in the Security Group for now.

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-04b70fa74e45c3917 (64-bit (x86)) / ami-0eac975a54dfee8cb (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Description

Canonical, Ubuntu, 24.04 LTS, amd64 noble image build on 2024-04-23

Architecture

64-bit (x86)

AMI ID

ami-04b70fa74e45c3917

Verified provider

▼ Instance type Info

Instance type

t2.large
Family: t2 2 vCPU 8 GiB Memory Current generation: true
On-Demand Windows base pricing: 0.1208 USD per Hour
On-Demand RHEL base pricing: 0.1528 USD per Hour
On-Demand SUSE base pricing: 0.1928 USD per Hour
On-Demand Linux base pricing: 0.0928 USD per Hour

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

▼ Summary

Number of instances Info

3

When launching more than 1 instance, consider EC2 Auto Scaling

Software Image (AMI)

Canonical, Ubuntu, 24.04 LTS, ...read more
ami-04b70fa74e45c3917

Virtual server type (instance type)

t2.large

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel Launch instance

▼ Configure storage Info Advanced

1x 20 GIB gp3 Root volume (Not encrypted)

Free tier eligible customers can get up to 30 GB of EBS General Purpose (SSD) or Magnetic storage

Add new volume

The selected AMI contains more instance store volumes than the instance allows. Only the first 0 instance store volumes from the AMI will be accessible from the instance

Click refresh to view backup information

The tags that you assign determine whether the instance will be backed up by any

Software Image (AMI)

Canonical, Ubuntu, 24.04 LTS, ...read more
ami-04b70fa74e45c3917

Virtual server type (instance type)

t2.large

Firewall (security group)

netflix-sg

Storage (volumes)

1 volume(s) - 20 GiB

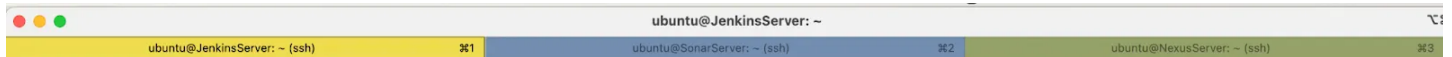
Cancel Launch instance

Review commands

Now 3 of the instances are ready and have named them as per our requirements. Please see below.

Instances (3) Info							
Find Instance by attribute or tag (case-sensitive)							
Instance state = running Clear filters							
	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
☐	Jenkins	i-0a0b9e17d450a14b3	Running	t2.large	Initializing	View alarms +	us-east-1e
☐	Sonar	i-005989c017d85a118	Running	t2.large	Initializing	View alarms +	us-east-1e
☐	Nexus	i-032b8f3c6f1267a05	Running	t2.large	Initializing	View alarms +	us-east-1e

All the servers are connected.



2. Install Jenkins: For installing Jenkins, please make sure Java is installed. As most of the organizations are moved to Java 17 we will be using the same as given below. Before that do update the Jenkins instance.

```
sudo apt update
```

```
sudo apt install openjdk-17-jre-headless
```

```
ubuntu@JenkinsServer:~$ java
Command 'java' not found, but can be installed with:
sudo apt install default-jre          # version 2:1.17-75, or
sudo apt install openjdk-17-jre-headless # version 17.0.10~6ea-1
sudo apt install openjdk-11-jre-headless # version 11.0.21+9-0ubuntu1
sudo apt install openjdk-19-jre-headless # version 19.0.2+7-4
sudo apt install openjdk-20-jre-headless # version 20.0.2+9-1
sudo apt install openjdk-21-jre-headless # version 21.0.1+12-3
sudo apt install openjdk-22-jre-headless # version 22~22ea-1
sudo apt install openjdk-8-jre-headless  # version 8u392-ga-1
ubuntu@JenkinsServer:~$
```

Install Jenkins LTS version, please follow the below.

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update
sudo apt-get install jenkins
```

Now Lets Start the Jenkins Service by enabling it.

```
sudo systemctl enable jenkins  
sudo systemctl start jenkins  
sudo systemctl status jenkins
```

If everything has been set up correctly, you should see an output like this:

Once Jenkins is installed, you will need to go to your AWS EC2 Security Group and open Inbound Port 8080, since Jenkins works on Port 8080.

Check in the browser — <EC2 Public IP Address:8080>

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Unlock Jenkins using an administrative password and install the suggested plugins.

Set an username password for Jenkins then “Save & Continue”.

Save and Finish.

Install Docker in the server

```
sudo apt install docker.io
```

Make sure Jenkins is able to run docker commands without root. For that we will use below. Please note this is not 100% secured as this can open several

vulnerability points.

```
sudo chmod 666 /var/run/docker.sock
```

3. Install Sonarqube in the Sonar Server

Login to the Sonar EC2 instance and then

```
sudo apt update
```

Install Docker

```
sudo apt install docker.io -y
```

```
sudo su
```

After the docker installation, we create a sonarqube container (Ensure to open 9000 ports in the security group).

```
docker run -d --name sonar -p 9000:9000 sonarqube:lts-community
```

Check if the Sonar container is running.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2686a6696adb	sonarqube:lts-community	"/opt/sonarqube/dock..."	About a minute ago	Up About a minute	0.0.0.0:9000->9000/tcp, :::9000->9000/tcp	sonar

```
root@SonarServer:/home/ubuntu#
```

Access <EC2 Public IP>:9000. Login with initial login, username: *admin* and password: *admin* and then you can set your own further step.

Now, Sonarqube is ready for use.

4. Install Nexus in the EC2 instance

Connect to the Nexus named EC2 instance then Install *Docker* first. Make sure the *8081* port is open in aws security group.

```
sudo apt update
```

```
apt install docker.io -y
```

Spin up the nexus docker container

```
docker run -d --name nexus -p 8081:8081 sonatype/nexus3
```

Let us go inside the container to fetch the password.

```
docker exec -it 65c918f8c842 /bin/bash
```

Access to the Nexus, by the browser.

<EC2 instance public IP:8081>

Sign in..

Lets fetch the initial password from the given file location in the pop-up message.

```
cd sonatype-work/nexus3/
```

```
ls
```

Next, set the new passwords. Enable anonymous success.

Nexus is completely set up now.

Exit, come out of the container.

```
exit
```

5. Install Plugins & Configure them

Sonar scanner, Nexus artifact uploader, docker, OWASP, eclipse temurine (helps to go for any version of jdk we need, we can configure) should be installed. Please see below

Configuration of tools:

JDK:

Manage Jenkins> Tools

Installing the *jdk11* as well just in case app doesnt work in the *jdk17*.

Sonarscanner:

Maven:

Dependency Check:

Docker:

6. Integrate Sonarqube & Nexus servers with Jenkins

6a. Integrate Sonarqube with Jenkins:

Generate a token in Sonarqube

Sonarqube Dashboard> Administration>Users>Below Token 3lines symbol>Genrate Token

Copy the Token.

Now go to Jenkins> Manage Jenkins> Credentials> Add Credentials

Add the token in the **Secret** field.

Add Docker Credentials as well.

Add DockerHub Username and Password under Global Credentials

Lets Configure the servers. Since we have added the tools we will be able to see Sonarqube server in the below path. Also we will be able to choose Server authentication token as we have added the token in the Global credentials.

Manage Jenkins> System> SonarQube servers

6b. Integrate Nexus with Jenkins:

To integrate Nexus we need to install a plugin called “Config File Provider” to get a specific option “*Manage Files*” in the Manage Jenkins Options.

Click on Manage Files> Add a new config> check on Global Maven settings.xml>
Provide a name “global maven”> Next

Configuring for “maven-releases”(usually used for Prod environment):

Copy the selected content from the script and press enter and paste the same.

Go to Nexus>Repositories>Maven Releases

This name will be used as ID, copy the name and go back to Jenkins>Managed files

Paste the name, and the username and password of Nexus.

Configuring for “maven-snapshots” (usually used for staging/dev environment).

Its done, Submit.

Need to do additional changes which is to be done in **pom.xml file in GitHub.**

Credentials of Nexus is configured in Jenkins but URL is not yet configured which we will be doing in This name will be used as ID, copy the name and go back to Jenkins>Managed files

Get Manisha Acharya's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Paste the name, and the username and password of Nexus.

Configuring for “maven-snapshots” (usually used for staging/dev environment).

Its done, Submit.

Need to do additional changes which is to be done in **pom.xml** file

Credentials of Nexus is configured in Jenkins but URL is not yet configured which we will be doing in **pom.xml** file.

For that we need to copy the URL from the Nexus and paste in repository section in **pom.xml file**.

Configuration complete.

7. Create Pipeline in Jenkins

Jenkins Dashboard>Create Job>Pipeline>Name the job>Ok

Write pipeline, have taken the “Hello World” template to get the format and update as per requirements.

1st stage for GIT checkout, have generated the step by using “pipeline syntax” option.

2nd & 3rd stage is for maven build, make sure to declare the maven & jdk version as we defined in the “Tools” configuration And a Test stage needs to be added in stage.

4th stage is ‘SonarQube Analysis’, make sure to declare the SONAR_SCANNER in the environments. Inside this \$SONAR_SCANNER/bin the project executable would be there.

5th Stage: Let's add OWASP dependency check and generate the pipeline syntax by selecting the below

Now we have to add the dependency check publisher by using Pipeline syntax.

6th Stage: Build Package

```
47  
48 stage('Build') {  
49     steps {  
50         sh "mvn package -DskipTests=true"  
51     }  
52 }  
53  
54
```

7th Stage: Deploying to Nexus

Before that make sure you have the plugin “*Pipeline Maven Integration plugin*” installed.

Then you will see the option in the Pipeline syntax

Build shows unstable, we need to look into it.

Because of this, the build is “Unstable”. After checking on the pipeline, there was a syntax error which is why parsing of .XML file was not happening. PFB the corrected pipeline code. Please make the changes.

You will see that in status, a graph will also be generated and Vulnerabilities.

8. Kubernetes Cluster setup

Take Ubuntu 24.04 t2.large instances with 20GB EBS volumes for k8s one master and the other two for worker.

Make sure to update the security group with the below ports open.

Name the servers by using the below commands

```
sudo hostnamectl set-hostname Master-node
```

```
sudo hostnamectl set-hostname worker-node1
```

```
sudo hostnamectl set-hostname worker-node2
```

Install the below in all 3 Nodes.

```
sudo apt update -y  
sudo apt install docker.io -y  
sudo service docker restart  
sudo curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.28/deb/Release.key | sudo
```

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs
```

```
sudo apt-get update  
sudo apt install -y kubeadm=1.28.1-1.1 kubelet=1.28.1-1.1 kubectl=1.28.1-1.1
```

Below command to be run only on Master Node.

```
kubeadm init --pod-network-cidr=192.168.0.0/16
```

The above-highlighted command needs to be executed in 2 worker nodes. Make sure you are on *root* user in both the worker nodes.

Here, coming out of root, and executing the given command in the below in *{Master node}*.

```
mkdir -p $HOME/.kube  
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Also execute the below commands in Master node.

Deploy Networking Solution (Calico)

Deploy Ingress Controller (NGINX)

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.25.6
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy.yaml
```

Calico yaml implements the Kubernetes Container Network Interface (CNI) as a plug-in and provides agents for Kubernetes to provide networking for containers and pods.

9. Create Namespace, Service, Role, Bind the Role to service,generate token

9a. Create the Namespace:

```
kubectl create namespace webapps
```

9b. Create Service:

```
vi sa.yml
```

```
apiVersion: v1  
kind: ServiceAccount  
metadata:  
  name: jenkins  
  namespace: webapps
```

execute the below command.

```
kubectl apply -f sa.yml
```

9c. Create Role:

```
apiVersion: rbac.authorization.k8s.io/v1  
kind: Role  
metadata:  
  name: app-role  
  namespace: webapps  
rules:  
  - apiGroups:  
    - ""
```

```
  - apps
  - autoscaling
  - batch
  - extensions
  - policy
  - rbac.authorization.k8s.io
resources:
  - pods
  - componentstatuses
  - configmaps
  - daemonsets
  - deployments
  - events
  - endpoints
  - horizontalpodautoscalers
  - ingress
  - jobs
  - limitranges
  - namespaces
  - nodes
  - pods
  - persistentvolumes
  - persistentvolumeclaims
  - resourcequotas
  - replicaset
  - replicationcontrollers
  - serviceaccounts
  - services
verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
```

9d. Bind the role to service account:

```
vi assign.yml
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-rolebinding
  namespace: webapps
roleRef:
  apiGroup: rbac.authorization.k8s.io
```

```
kind: Role
name: app-role
subjects:
- namespace: webapps
  kind: ServiceAccount
  name: jenkins
```

Now the service account 'jenkins' is having the access to create application, pod etc. or whatever is defined in the role is provided.

9e. Generate token using service account in the namespace

```
vi sec.yml
```

```
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: mysecretname
  annotations:
    kubernetes.io/service-account.name: jenkins
```

As there is no namespace mentioned in the yaml file, while creating the secret we should make sure it is created in our namespace "webapps".

```
kubectl apply -f sec.yml -n webapps
```

To view the secret created, below is the token which will be used for authentication.

10. Build & tag the Docker image

By using the pipeline syntax, generated the command for docker registry login.

As the dockerfile doesn't exist inside the root folder in repo, hence provided the file location in docker build execution.

11. Integrate Trivy with Jenkins

— TRIVY INSTALLATION STEPS —

```
sudo apt-get install wget apt-transport-https gnupg lsb-release  
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo apt-  
echo deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main  
sudo apt-get update  
sudo apt-get install trivy
```

Trivy installation is complete, we need to add the stage of trivy scan of the docker image in the Jenkins pipeline.

12. Create a stage to push the Docker image to DockerHub

13. Create a stage to deploy the application to K8S cluster

Firstly install the below plugins.

Jenkins Dashboard>Manage Jenkins> Plugins

Kubernetes

Kubernetes CLI

Back to Master Node:

By using the below command, we can get the token for the secret we created.

```
kubectl -n webapps describe secret mysecretname
```

Get the *kubeconfig file* because it contains all the information about the K8s cluster which we will be using in Jenkins integration.

Back to Jenkins Pipeline, write the final stage by using Pipeline syntax:

Copy the token we generated previously in the Master node, use it in the credentials field below.

Added the token, now select the same in credentials.

Copy the server details and paste in K8S server end point.

Generate the pipeline syntax.'

Build Now.

Looks like Build failed. Lets look into the console output

Have given 666 permissions to docker.sock.

Now the docker-related error is resolved. Build again.

But again the build fails at then end stage. lets dig into it.

kubectl is not found, kets check in the Jenkins server.

Let us re-build the pipeline.

This time build is successful, let's access the Ekart from the given port in the console output.

Was still unable to access the application, after research got to know that we have to change type from "Loadbalancer" to "NodePort" in the *deployment-service.yml* file in github.

Lets login as admin(username, password), and try to access.

Application deployed in K8S cluster successfully.

Application deployed in K8S cluster successfully.

Try out this project and get hands on various DevOps tool and add the project into your Resume. I have shared the complete live demonstration with all the error solving for build failures.

See you soon on an another blog.

Manisha Acharya —

[Follow](#)

Written by Manisha Acharya

44 followers · 12 following

I'm a DevOps enthusiast turned traveler, leaving behind a recruiter's desk to dive into the world of continuous learning.

No responses yet



Write a response

What are your thoughts?

More from Manisha Acharya



Manisha Acharya

Deploying a Secure Hotstar Clone AWS EKS Cluster with DevSecOps Principles

Imagine you want to build a copy of a streaming service like Hotstar. This blog explains how to set it up on Amazon's cloud service (AWS)...

May 20, 2024  14



Manisha Acharya

What is Dockerfile

A Dockerfile is a text document that contains a set of instructions for building a Docker image. Docker is a platform for developing...

Mar 18, 2024



Manisha Acharya

Automating ChatGPT clone Deployment with EKS, Terraform, and Jenkins

In today's rapidly evolving tech landscape, automating application deployment and monitoring is crucial for maintaining efficiency and...

Jun 17, 2024





Manisha Acharya

Basic Linux Commands

In my #90daysOfDevops journey as per the syllabus let's get into Linux Basic commands.

Oct 30, 2023 🖱 2



See all from Manisha Acharya

Recommended from Medium



In AWS Tip by Maximilian Oliver

Deploying Secure APIs with API Gateway, Cognito, and IAM in AWS

A Practical Guide to Building, Securing, and Scaling REST APIs Without Managing Servers.



Aug 27 🖱 7





techWithNeer

Stop Using Microservices—This Architecture Performs Better at Scale

When everyone is busy breaking down their systems into a hundred services, we decided to do the opposite. Here's why it worked.



Jul 11



478



50



In Javarevisited by Saquib Aftab

Understanding MCP and How to Create a MCP Server using Spring Boot?

MCP stands for Model Context Protocol. It is an open-source standard which was introduced in November 2024 by Anthropic.

🌟 6d ago 🤝 48



Pudding Entertainment

GitHub Actions: Spring Boot Application Build and Deployment

Get started with GitHub Actions with this simple tutorial

May 18 🤝 2

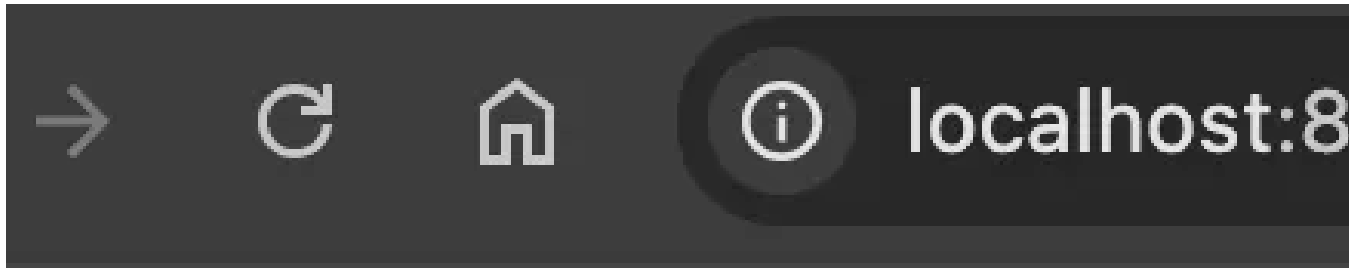


Code With Sunil | Code Smarter, not harder

Unit Testing in Java with Mockito and JUnit 5: A Beginner's Guide

If you are not a Member — Read for free here

★ Sep 4 🖱 7 💬 2



World!

 Ansgar Mikel Nell

Spring Boot Security, Step By Step — Part 1: Authentication

*Note: Part 2 coming soon

Jul 17 🖱 12



See more recommendations